

Academic Question Answering System Using RAG

A Retrieval-Augmented Generation Approach for Accurate, Source-Cited Academic
Answers

Prepared by	Vishesh Maurya
Department	Computer Science and Engineering
Institution	ABES Engineering College, Ghaziabad
Academic year	2026–2027

Artificial Intelligence • Information Retrieval • Natural Language Processing

Abstract

The Academic Question Answering System Using Retrieval-Augmented Generation (RAG) is an AI-powered application designed to answer academic questions using information retrieved from a collection of educational PDF documents. It addresses limitations of conventional AI chatbots, which may produce incorrect, outdated, or unsupported information.

The proposed system extracts text from PDFs, divides the content into smaller chunks, converts those chunks into vector embeddings, and stores them in a vector database. When a user submits a question, the system retrieves relevant passages and provides them to a Large Language Model (LLM), which generates an answer grounded in the retrieved material. A key feature is source traceability through PDF and page-number citations, allowing users to verify the answer against the original material.

The initial implementation focuses on educational resources related to Artificial Intelligence and RAG. It can be extended to additional subjects, larger document collections, and more advanced evaluation methods.

Contents

Section	Title
1–3	Introduction, Project Title, Problem Statement
4–6	Aim and Objectives, Scope, Methodology
7–9	Technology Stack, System Architecture, Citation Mechanism
10–13	USP, Feasibility, Requirements, Dataset
14–18	Testing, Current Status, Limitations, Future Enhancements, Outcomes
19–20	Conclusion and References

1. Introduction

Students often study from lecture notes, textbooks, research papers, and other PDF resources. Finding a specific explanation across multiple documents can be time-consuming. Traditional keyword search may fail to identify relevant passages when the wording of a question differs from the wording in a document.

General-purpose AI chatbots can explain academic topics, but their answers may not be restricted to a student's supplied study material. Retrieval-Augmented Generation offers an alternative by searching a knowledge base first and using relevant passages as context for answer generation. This project applies RAG to academic question answering, emphasizing relevance, grounded generation, and verifiable citations.

2. Project Title

Academic Question Answering System Using Retrieval-Augmented Generation (RAG) with PDF-Based Source Citations

3. Problem Statement

Students need a reliable way to obtain answers from academic documents without manually searching lengthy PDFs. Existing approaches present several challenges:

- **Time-consuming search:** Finding relevant information across study documents requires considerable effort.
- **Limited semantic search:** Basic keyword search may miss relevant passages expressed using different terminology.
- **Unverified AI responses:** Language models can generate plausible but unsupported information.
- **Lack of traceability:** Answers may not identify the document or page supporting a statement.

- **Disconnected resources:** Information spread across PDFs is difficult to access through a single question-answering interface.

Therefore, the problem is to develop a document-grounded question-answering system that retrieves relevant passages from academic PDFs, generates answers based on those passages, and provides source and page references for verification.

4. Aim and Objectives

4.1 Aim

To develop an AI-based academic question-answering system that uses RAG to provide relevant, context-grounded answers from PDF study materials with verifiable document and page citations.

4.2 Objectives

- Extract readable text from academic PDFs while preserving page-level information.
- Preprocess and divide the extracted text into manageable chunks.
- Generate vector embeddings representing the semantic meaning of each chunk.
- Store and index embeddings for efficient similarity-based retrieval.
- Retrieve relevant passages in response to user questions.
- Generate answers using an LLM grounded in the retrieved context.
- Display source PDF names and page numbers.
- Evaluate correctness, retrieval relevance, faithfulness, and citation accuracy.
- Reduce unsupported responses by acknowledging insufficient evidence.
- Allow additional documents and academic subjects to be added later.

5. Project Scope

5.1 Included in the Initial Scope

- Selectable-text academic PDFs, initially focused on AI and RAG.
- PDF text extraction, cleaning, and chunking with overlap.
- Embedding generation and vector-based retrieval.
- LLM-powered answer generation.
- PDF and page-level citation support.
- Evaluation and functional testing.

5.2 Potential Future Extensions

- OCR for scanned PDFs and support for additional document formats.
- More academic subjects and larger document collections.
- Conversation history, follow-up questions, and user document management.
- Highlighting cited passages and advanced evaluation dashboards.

The initial version is intended as an academic prototype rather than a full-scale commercial learning platform.

6. Proposed Approach and Methodology

Phase A — Knowledge-Base Preparation

Step	Activity	Description
1. PDF collection	Collect academic resources	Gather relevant PDFs about RAG, AI, embeddings, language models, and information retrieval.
2. Text extraction	Extract document text	Use a PDF processing library and preserve the source filename and original page number.
3. Cleaning and chunking	Prepare text passages	Remove extraction noise where appropriate and split text into smaller, overlapping chunks.
4. Embedding generation	Create vectors	Convert each chunk into a numerical representation using an embedding model.
5. Vector storage	Index passages	Store embeddings, chunk text, and source metadata in a vector database such as ChromaDB.

Initial chunking configuration: approximately 250 tokens per chunk with 60 tokens of overlap. These are starting parameters and should be adjusted based on retrieval and citation evaluation.

Phase B — Question Answering

Step	Activity	Description
6. User question	Accept input	The user asks a question about the indexed academic material.
7. Query embedding and retrieval	Search semantically	Convert the question to an embedding and retrieve relevant passages.
8. Context construction	Prepare evidence	Collect relevant text chunks and metadata such as PDF names and page numbers.
9. Answer generation	Use an LLM	Generate a response grounded in retrieved context and instruct the model not to invent unsupported facts.
10. Answer and citations	Present result	Display the answer and source references; indicate when sufficient evidence is unavailable.

7. Technology Stack

Technology	Role in the project
Python	Core language for PDF processing, chunking, embeddings, retrieval, model calls, and evaluation.
PyMuPDF	Extracts text and page-level content from selectable-text PDFs.
Sentence Transformers	Converts text and questions into vectors for semantic retrieval; document the actual model used.
ChromaDB	Stores vectors, text chunks, and source metadata for similarity search.
Google Gemini API	Generates natural-language answers from retrieved context, if this is the configured provider.
LangChain (optional)	Can support recursive text splitting and orchestration if included in the implementation.
Streamlit (optional)	Can provide a simple question-and-answer interface; Flask or FastAPI are alternatives.
Git and a Python environment	Version control, dependency isolation, and reproducible development.

Note: Optional tools are recommendations, not confirmation that every library is installed. The final submission should list the actual dependencies and model names from the project configuration.

8. System Architecture

The architecture separates document ingestion, retrieval, answer generation, and presentation. Indexing runs when documents are added or updated; the question-answering pipeline runs whenever a user asks a question.

Pipeline	Sequence
Knowledge-base preparation	Academic PDFs → Text extraction → Cleaning and chunking → Attach source/page metadata → Embedding model → Vector database
Question answering	User question → Query embedding → Similarity search → Relevant chunks → Context with citations → LLM answer generation → Answer and page references → User interface
Evaluation	Evaluation dataset → Retrieval checks and answer/citation evaluation

8.1 Architecture Components

Component	Responsibility
Document ingestion	Loads academic PDFs into the processing pipeline.
Text extraction	Extracts content and associates it with the original page.
Chunking and metadata	Creates text segments and preserves source information.
Embedding model	Converts chunks and user questions into vectors.
Vector database	Indexes and retrieves semantically similar passages.
Retrieval module	Selects relevant evidence for a question.
LLM module	Generates a context-grounded answer.
Citation module	Associates retrieved evidence with source PDF and page references.
Evaluation module	Measures retrieval quality, answer correctness, faithfulness, and citation accuracy.
User interface	Displays questions, answers, and source references.

9. Citation and Grounding Mechanism

Citation accuracy is the primary differentiating requirement. Each chunk should retain metadata that allows the system to trace the passage to its source.

Metadata field	Purpose
document_id	Unique identifier for the source document.
source	Original PDF filename.
page_number	Original page from which the passage was extracted.
chunk_id	Unique identifier for a text chunk.
text	Extracted passage used in retrieval and generation.
section (optional)	Chapter or heading information, where available.

- Citations must come from retrieved source metadata rather than being generated freely by the LLM.
- Each cited passage should support the associated claim.
- Multiple sources should be shown when multiple documents support the answer.
- The system should distinguish missing evidence from a supported answer.
- Citation correctness must be evaluated separately from answer fluency.

Illustrative response format: An answer explains RAG and lists “rag_fundamentals.pdf — Page 4.” This is only a format example; source references must be verified against the actual dataset before use.

10. Novelty and Unique Selling Proposition (USP)

The project's USP is academic question answering with traceable, page-level evidence from user-provided study materials.

Feature	Keyword search	General AI chatbot	Proposed RAG system
Searches supplied PDFs	If indexed	Not necessarily	Yes

Feature	Keyword search	General AI chatbot	Proposed RAG system
Semantic similarity	Limited in basic search	Generally	Yes, through embeddings
Natural-language answers	Typically no	Yes	Yes
Uses retrieved passages	Depends on design	Not necessarily	Core design
PDF and page references	Depends on implementation	Not guaranteed	Core requirement
Evaluation against a document set	Possible	Not inherent	Yes

- **Source-grounded responses:** Answers are based on retrieved academic content.
- **Page-level traceability:** Students can locate supporting passages in the original PDF.
- **Semantic retrieval:** The system can find conceptually relevant text even when wording differs.
- **Domain-specific knowledge base:** Answers can be restricted to selected study materials.
- **Evidence-aware responses:** The system can report when documents do not provide sufficient evidence.
- **Modular design:** Processing, retrieval, generation, and evaluation can be improved independently.

The USP is not simply the use of an LLM or vector database, but their integration into a verifiable academic workflow.

11. Feasibility Analysis

11.1 Technical Feasibility

The project is feasible as an academic prototype because it uses established Python libraries and accessible AI technologies. A small collection of selectable-text PDFs avoids the extra complexity of OCR. Main challenges include retrieval relevance, page metadata, LLM integration, and citation validation.

11.2 Economic Feasibility

Resource	Consideration
Python libraries	Generally available without a licence fee.
ChromaDB	Open-source vector storage option.
Embedding model	Local execution may avoid per-request API charges.
Gemini API or other LLM API	Subject to current pricing, model availability, quotas, and rate limits.
Development computer	An existing computer may be sufficient for a small dataset.
Hosting	Optional for a local demonstration; may incur costs if deployed.

11.3 Operational Feasibility

The intended workflow is simple: prepare PDFs, index documents, ask a question, and inspect the answer with its citations. Users do not need to understand the internal embedding or retrieval algorithms.

11.4 Schedule Feasibility

Stage	Work package
1. Dataset preparation	PDF collection, extraction checks, metadata validation.
2. Indexing and retrieval	Chunking, embeddings, vector storage, retrieval checks.
3. Answer generation	LLM integration, prompt design, citation formatting.
4. Evaluation and submission	Test questions, error analysis, documentation, and demonstration.

These are sequential work packages rather than a claim about the actual completion time.

12. Functional and Non-Functional Requirements

12.1 Functional Requirements

- Process supported academic PDF files.
- Extract text while preserving source page information.
- Create and store text embeddings.
- Accept natural-language questions.
- Retrieve relevant passages from the indexed collection.
- Generate answers using retrieved context.
- Display source PDF names and page references.
- Handle questions without adequate supporting material.
- Support evaluation with a separate set of test questions.

12.2 Non-Functional Requirements

- **Accuracy:** Answers should be consistent with retrieved evidence.
- **Traceability:** Citations should point to valid source locations.
- **Performance:** Response times should be reasonable for the target dataset.
- **Maintainability:** Processing, retrieval, and generation should be separable.
- **Scalability:** The design should allow additional documents and subjects.
- **Security:** Keep API credentials outside source code, such as in environment variables.
- **Reliability:** Handle API errors, missing files, empty extraction results, and unsupported questions.
- **Reproducibility:** Document model names, chunking, retrieval settings, and generation settings.

13. Dataset Description

The initial dataset consists of approximately 1–5 selectable-text PDFs related to AI and RAG. These documents form the knowledge base used for retrieval and answer generation.

Field	Description
Document ID	Unique identifier for each PDF.
File name	Original PDF name.
Page number	Original page associated with extracted text.
Extracted text	Text obtained from the document.

Field	Description
Chunk ID	Identifier for each processed text segment.
Chunk text	Text passage supplied to retrieval.
Embedding	Vector representation of the passage.
Source metadata	Document and page information needed for citations.

13.1 Evaluation Dataset

The evaluation set should be separate from the document collection. It should contain representative questions, expected answers or key answer points, and the relevant source document and page.

- **Direct factual questions:** Answer is stated explicitly in one passage.
- **Conceptual questions:** Answer requires understanding an explanation.
- **Multi-passages questions:** Answer combines evidence from multiple passages.
- **Paraphrased questions:** Question wording differs from the document.
- **Out-of-scope questions:** The answer is not supported by indexed material.
- **Citation-focused questions:** Test whether the source and page support the answer.

14. Testing and Evaluation Strategy

Evaluation should measure the retrieval stage, answer-generation stage, and citation stage separately.

Evaluation area	What it measures	Suggested measures
Retrieval	Whether retrieved passages contain the needed evidence.	Recall@K, Precision@K, Mean Reciprocal Rank where appropriate.
Answer generation	Whether responses answer the question and remain faithful to retrieved context.	Correctness against reference answers, completeness, faithfulness review.
Citation quality	Whether citations identify the correct source/page and support the claim.	Citation validity rate, support rate, page-reference accuracy.

Test ID	Scenario	Expected result
TC-01	Question explicitly covered by a PDF	Relevant answer with supporting citation.
TC-02	Question uses different wording	Semantically relevant passages are retrieved.
TC-03	Question requires multiple passages	Answer combines evidence and identifies sources.
TC-04	Topic absent from the PDFs	System indicates insufficient supporting information.
TC-05	Verify a page reference	Referenced page exists and supports the answer.
TC-06	Invalid or empty question	Input is handled without crashing.
TC-07	LLM API failure	Error is handled and reported clearly.
TC-08	Add a new PDF	Content becomes searchable after processing and indexing.

The final report should include the number of test questions, evaluation procedure, actual metric values, observed errors, and limitations. Do not claim a numerical accuracy rate unless it has been measured on the evaluation dataset.

15. Current Project Status

Based on the latest progress reported during project development, evaluation completed successfully and automated tests passed. The following status summary should be confirmed against the current codebase before final submission.

Area	Reported status	Note
PDF text extraction	Completed	Confirm text and page metadata in the current dataset.
Text chunking	Configured	250-token target and 60-token overlap were discussed.
Answer pipeline	Working	Verify against representative questions.
Evaluation run	Success	This does not establish perfect accuracy.
Automated tests	Passed	Only the covered cases are verified.

Passing evaluation and functional tests indicates that the tested workflow is operating as expected. It does not establish perfect answer accuracy or citation reliability; these should be reported using measured results.

16. Limitations

- **Limited document coverage:** A small PDF collection cannot answer every academic question.
- **Extraction errors:** Complex layouts, tables, equations, and unusual formatting may be extracted incorrectly.
- **Retrieval errors:** The most relevant passage may not appear among the retrieved chunks.
- **Hallucinations:** An LLM may introduce unsupported details even when context is supplied.
- **Citation errors:** A relevant passage may not support the complete generated answer.
- **API dependency:** Cloud-based generation depends on connectivity, quotas, and service reliability.
- **Chunk-size trade-offs:** Small chunks may omit context; large chunks may reduce precision.
- **Evaluation limitations:** A small test set may not generalize to other subjects or collections.

Potential mitigations include improved extraction, retrieval tuning, evidence-aware prompting, citation checks, and a broader evaluation dataset.

17. Future Enhancements

- Support scanned PDFs through OCR.
- Hybrid retrieval combining semantic and keyword search.
- Reranking retrieved passages to improve relevance.
- Clickable citations that open the original PDF at the referenced page.
- Highlighting the exact passage used to generate an answer.
- Support for tables, equations, and structured academic content.
- Additional embedding models and configurable LLM providers.
- Conversation history and contextual follow-up questions.
- Document upload and knowledge-base management.
- Larger benchmark datasets and automated citation evaluation.
- Deployment as a web application for student use.

18. Expected Outcomes

- Process a collection of academic PDFs.
- Retrieve relevant information using semantic similarity.
- Generate natural-language answers using an LLM.
- Preserve document and page metadata throughout the pipeline.
- Provide source references that users can independently verify.
- Handle questions not sufficiently supported by the knowledge base.
- Support repeatable evaluation of retrieval, answer generation, and citations.

The project demonstrates practical applications of embeddings, vector databases, information retrieval, prompt design, and LLM integration.

19. Conclusion

The Academic Question Answering System Using RAG applies retrieval-augmented generation to make academic information more accessible and verifiable. By integrating PDF text extraction, chunking, embeddings, vector search, and language generation, the system provides a structured approach to answering questions from a defined collection of study materials.

Its main contribution is the emphasis on grounded answers with document-level and page-level citations, rather than generating responses without clear supporting evidence. This makes the approach relevant to academic learning, revision, and document-based research. Further improvements in retrieval quality, citation validation, evaluation coverage, and user-interface design can increase reliability and practical usefulness.

20. References and Further Reading

1. Lewis et al. (2020), “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.”

Foundational research on combining retrieval and generation.
<https://arxiv.org/abs/2005.11401>

2. Reimers and Gurevych (2019), “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks.”

Background on sentence embeddings and semantic similarity.
<https://arxiv.org/abs/1908.10084>

3. PyMuPDF Documentation

PDF text extraction and page processing.
<https://pymupdf.readthedocs.io/>

4. Chroma Documentation

Vector storage, collections, and similarity search.
<https://docs.trychroma.com/>

5. Google Gemini API Documentation

Model integration and generation API documentation.
<https://ai.google.dev/gemini-api/docs>

6. Sentence Transformers Documentation

Embedding models and semantic search.
<https://sbert.net/>

Submission note: Before submitting, add actual application screenshots, the current project directory structure, sample questions and answers, measured evaluation results, and verified citations from the real dataset. These materials distinguish implemented features from proposed enhancements.